

Mini Project 3 — Real-Time Book Recommendation System

DSAI 427 Big Data Analytics

Mohamed Hozien *Partner Name*

May 2026

1 Executive Summary

We built an end-to-end big-data pipeline that learns book preferences from the Kindle Reviews dataset using Spark MLlib's ALS, streams new interactions through Kafka, computes 30-second sliding-window analytics with Spark Structured Streaming, and serves hybrid Top-5 recommendations dynamically. The system is augmented with a Streamlit dashboard and meets a sub-five-second end-to-end latency target.

Domain: Books (Kindle Reviews, ~982K ratings).

Focus: Real-Time Intelligence (trending detection + alerts) with a slice of System Optimization for the latency bonus.

Bonuses pursued: Streamlit dashboard (+2), latency <5 s (+1).

2 System Architecture

```
historical CSV → ALS Trainer → als_model/ → Streaming app
Producer → Kafka(interactions, 3p) → Spark Structured Streaming → {recs,
                                                                    alerts} topics + parquet sinks
Streamlit dashboard polls parquet sinks every 2s.
```

3 Dataset and Justification

The dataset is the Kindle Reviews release on Kaggle ([bharadwaj6/kindle-reviews](https://www.kaggle.com/bharadwaj6/kindle-reviews)). It contains ~982K rows with columns `reviewerID`, `asin`, `overall`, `unixReviewTime`, mapping cleanly to the required (`user_id`, `item_id`, `rating`, `timestamp`) schema and exceeding the 500K-row threshold by nearly 2×. Distributed processing is required because the user-item matrix is sparse and high-dimensional, ALS factor updates involve repeated joins that benefit from Spark's shuffle infrastructure, and sustained streaming throughput (we target 50–80 ev/s with bursts) outpaces single-process tooling. Data challenges include duplicate (`user`, `item`) pairs (we keep the latest by timestamp), out-of-range ratings (clipped to [1, 5]), and free-text fields that complicate CSV parsing (`multiLine=true`).

4 Machine Learning Component

4.1 Preprocessing

We drop rows with null `reviewerID`, `asin`, or `overall`, restrict ratings to [1, 5], and deduplicate (`user`, `item`) pairs by keeping the latest `unixReviewTime`. `StringIndexer` converts `reviewerID` and `asin` into integer keys that ALS expects.

4.2 Training

We train ALS with `coldStartStrategy="drop"`, `nonnegative=True`, and an 80/20 random split (seed 42). The baseline configuration is `rank=10`, `regParam=0.1`, `maxIter=10`.

4.3 Evaluation and Tuning

RMSE is computed on the held-out 20%. If RMSE exceeds 1.5 we run a $2 \times 2 \times 2$ grid over `rank` $\in \{10, 20\}$, `regParam` $\in \{0.05, 0.1\}$, `maxIter` $\in \{10, 15\}$ and keep the best configuration. *Insert final RMSE table here after running `train_als.py`.*

5 Streaming Component

5.1 Kafka Setup

Kafka 2.12-3.7.0 runs in KRaft single-node mode from a portable folder (`$KAFKA_DIR/bin/kafka-server-start`). The `interactions` topic uses **3 partitions** keyed by `user_id%3`; this preserves per-user event ordering while balancing across the three Spark cores. `recommendations` and `alerts` use 1 partition each since they are downstream sinks consumed by the dashboard.

5.2 Spark Structured Streaming

- **Source:** `kafka.subscribe = interactions`, `maxOffsetsPerTrigger = 2000`, `startingOffsets = latest`.
- **Parsing:** `from_json` with an explicit schema; rows whose `evt` struct is `null` are flagged malformed and dropped. A defensive `filter` also drops out-of-range ratings.
- **Watermark:** `event_time`, **1 minute**.
- **Window:** 30-second size, 10-second slide, computed on `event_time`.

5.3 Window Analytics and Custom Metric

Per-window aggregates are: average rating per item, interaction count per user, and our custom **trending score**:

$$trending(i, w) = count(i, w) \cdot \overline{rating}(i, w) \cdot \exp\left(-\frac{\Delta t}{5}\right)$$

where Δt is the gap between the window end and the latest event for item i in minutes. The exponential decay rewards **recent** bursts within a window while still weighting items by quality and volume; a flat count-only metric would over-rank stale items that happened to peak earlier in a sliding window.

6 ML + Streaming Integration

At streaming-app start we read the saved ALS factor parquets into the driver process and use them for per-batch Top-5 generation. Each micro-batch:

1. Collects distinct user IDs.
2. Computes ALS scores via a NumPy dot product against the broadcast item-factor matrix.
3. Blends with the latest trending state: $score = 0.7 \cdot \widehat{ALS} + 0.3 \cdot \widehat{trending}$, both normalized to $[0, 1]$.
4. Writes the Top-5 to the `recs/` parquet sink and the `recommendations` Kafka topic.

Cold-start users (not seen in batch training) fall back to the current top-trending list.

7 Alert System and Late Data Handling

7.1 Alert Rules

- **Trending item:** `avg_rating > 4.5` AND `count ≥ 10` in the same window.
- **User burst:** a single user emits `≥ 20` events in one window (e.g. a bot or a power user spree).

Alerts go to console, the `output/alerts/` parquet sink, and the `alerts` Kafka topic.

7.2 Late Data

With `watermark = 1 minute` and `outputMode=append`, records arriving later than the watermark are silently dropped. We chose `append` (rather than `update`) because alert freshness and bounded state outweigh the benefit of re-emitting late updates.

8 Latency Measurement (Bonus)

The producer stamps `event_time_ms` at send time. The streaming app's `foreachBatch` sink subtracts that from the Kafka `ingestion` timestamp and writes per-record latency to `output/latency/`. Knobs applied to stay under 5s: `trigger=ProcessingTime("1 second")`, `maxOffsetsPerTrigger=2000`, Kryo serializer, `spark.sql.shuffle.partitions=8`, factor matrices kept in driver memory (no per-batch I/O). *Insert measured p50/p95 histogram here after the 5-minute run.*

9 Results

Replace placeholders with the values produced by your run:

- ALS RMSE (final): `<value>`
- Average events/sec sustained: `<value>`
- Latency p50 / p95: `<ms> / <ms>`
- Total alerts triggered in a 5-minute demo: `<value>`

10 Differentiation

Books domain (not the default movies); custom time-decayed trending score; hybrid recommender mixing ALS and trending with a normalized 70/30 blend; cold-start fallback to top-trending; per-user partition keying that aligns the 3 Kafka partitions with 3 Spark cores; sub-five-second latency budget measured per record.

11 Challenges and Lessons Learned

- Aligning Spark 4.1.1 (Scala 2.13) with the matching `spark-sql-kafka-0-10_2.13` connector — mismatched Scala builds silently fail at JSON parsing.
- Streaming state can grow unbounded; we cap the trending state map and rely on watermark-driven aggregation cleanup.
- ALS training is a one-shot batch job; integrating it with streaming requires either re-loading factors (chosen) or running predictions on executors with broadcast variables.

12 Reproducing the Pipeline

See README.md. Short version:

```
export KAFKA_DIR=~/.kafka_2.12-3.7.0
bash infra/start_kafka.sh
bash infra/create_topics.sh
python src/download_dataset.py
python src/train_als.py
python src/kafka_producer.py --mode synthetic --rate 80 &
spark-submit --packages org.apache.spark:spark-sql-kafka-0-10_2.13:4.1.1 \
    --master "local[3]" src/streaming_app.py &
streamlit run src/dashboard.py
```